

Universidade Federal da Paraíba (UFPB)

Alunos: (Arthur Vieira - 20210094400) e (André Lopes - 20210066881)

Tópicos Avançados em Ciência da Computação XI

Relatório Atividade 04 - Gostoteca

Link do Github com o código: <https://github.com/THURRnado/Gostoteca>

Vídeo apresentação: https://www.youtube.com/watch?v=_3hwFhDraFM

Visão geral do projeto

O projeto Gostoteca implementa uma aplicação web em Django para catalogar preferências de mídia (jogos, filmes, livros) com uma camada social baseada em reviews (nota e opinião) e gerenciamento de contas de usuários. A arquitetura concentra um app `usuarios` com modelo de usuário customizado e autenticação completa, e um app `catalogo` com os modelos `Item` e `Review`, CRUD autorizado e contagem de reviews no catálogo.

Uso de agentes no processo de desenvolvimento

Papel dos agentes e subagentes

O desenvolvimento foi conduzido com apoio intensivo de agentes, incluindo um agente principal (Claude Code, modelo sonnet 4.6) e subagentes especializados em implementação passo-a-passo via "superpowers" com planos TDD. Esses agentes foram usados tanto para gerar código (views, forms, templates, modelos) quanto para escrever specs de design, planos de implementação e documentação em Markdown, sempre estruturados em tarefas com testes que falham primeiro e depois passam.

Estrutura de trabalho com agentes

O fluxo padrão foi: (1) analisar o documento da atividade e registrar decisões de produto e arquitetura, (2) escrever um spec de design por fatia (login/cadastro, catálogo de itens, reviews

e exclusão de conta), e (3) gerar um plano TDD detalhado para agentes executores, com passos numerados e comandos esperados. Cada fatia do projeto (autenticação, item e catálogo, review e exclusão de conta) teve arquivos distintos de design e plano (*-design.md e *.md), que guiaram os subagentes em ciclos curtos de implementação e revisão.

Evolução por fatias com agentes

Fatia 1: Telas de login, cadastro e logout

A primeira fatia focou em entregar telas de cadastro, login e logout no app usuarios, com sete testes automatizados previstos e dez entregues ao final. O design escolheu LoginView e LogoutView do Django para reduzir código próprio e garantir comportamento testado de sessão, next e proteção contra session fixation, complementados por uma CadastroView baseada em CreateView com UsuarioCreationForm herdando de UserCreationForm.

Os agentes documentaram as decisões divergentes em relação ao documento original (uso de Tailwind em vez de Bootstrap, email obrigatório e único via modelo, ausência de auto-login após cadastro) e registraram riscos como a dependência de rede do Play CDN. O plano de implementação em 2026-07-14-login-cadastro.md definiu uma sequência rígida: escrever testes que falham, rodar manage.py test, implementar o mínimo, rodar novamente e cometer apenas pelo usuário humano, pois os agentes foram instruídos explicitamente a não executar comandos de git.

Fatia 2: Modelo Item, catálogo, CRUD autorizado

A segunda fatia tratou do modelo Item e do catálogo, fechando o critério "rotas protegidas exigem login" da seção 5 do documento da atividade. O design em 2026-07-14-catalogo-item-design.md definiu que qualquer usuário logado pode criar itens, enquanto apenas o dono ou um admin (via is_staff) pode editar ou excluir, com um DonoOuAdminMixin baseado em UserPassesTestMixin e LoginRequiredMixin em todas as views.

Os agentes implementaram o modelo Item com TextChoices para o tipo (Jogo, Filme, Livro), campos opcionais para criador, descrição e capa, e uma FK criador para o modelo de usuário

customizado, garantindo integridade via `AUTH_USER_MODEL`. Também configuraram upload real de imagens com `ImageField`, dependência de `Pillow`, `MEDIA_URL/MEDIA_ROOT` e `static()` em `urls.py`, além de formulários com `FormEstilizado` para aplicar classes `Tailwind` a widgets nativamente sem classe.

Fatia 3: Review, autorização assimétrica e exclusão de conta

A fatia final entregou o modelo `Review` com CRUD autorizado, contagem no catálogo, e exclusão da própria conta, fechando o último CRUD pedido e deixando claros dois critérios que não seriam implementados (remoção de item em uso bloqueada e comentários aninhados). O design em `2026-07-14-review-e-conta-design.md` registrou que `Review` substitui a entidade "preferência" descrita no documento, com nota obrigatória de 0 a 10, opinião opcional e restrição de uma review por usuário por item via `UniqueConstraint`.

Os agentes generalizaram os mixins de autorização em `catalogomixins.py`, criando `DonoMixin` parametrizável via `campodono` e mantendo `DonoOuAdminMixin` para cenários em que admins podem apagar, mas não editar reviews alheias, alinhando a regra de negócio "moderar removendo, não reescrevendo a opinião de alguém". A exclusão de conta foi implementada com uma `ContaDeleteView` que opera exclusivamente sobre `request.user`, exibe em tela os números do CASCADE (quantos itens e quantas reviews de terceiros serão apagados), executa `logout` antes do `delete` e redireciona para o login.

Experiência do grupo com o desenvolvimento assistido por agentes

Tomada de decisão e registro de divergências

Ao longo da sessão, o grupo usou agentes para apoiar decisões de nome de projeto, divisão em apps, modelagem de usuário, identidade visual e fatiamento do escopo. Essas decisões foram sistematicamente registradas em `2026-07-14-resumo-da-sessao.md`, que serve como memória técnica do processo, destacando a importância de corrigir o documento da atividade ou registrar divergências explícitas no relatório final.

A experiência mostrou que agentes são valiosos para enumerar alternativas e impactos (por exemplo, escolha de `is_staff` em vez de `Group` para `admin`, adoção de `CASCADE` ao invés de bloqueio de remoção em uso, eliminação do modelo `Comentario`), mas que o controle das decisões de produto e das implicações para critérios de aceite precisa permanecer com o grupo humano.

Interações, correções e descobertas técnicas

O resumo da sessão relata vários episódios em que agentes e subagentes detectaram inconsistências entre planos e realidade (contagem de testes errada, testes dependendo de rotas ainda não criadas), levando a paradas conscientes e correções antes de prosseguir. Também documenta descobertas técnicas como o bug do `ul` dentro de `p` nos templates de senha, a exigência de `LogoutView` aceitar apenas `POST` em Django 5, e a necessidade de `transaction.atomic` em testes que forcem `IntegrityError` em `UniqueConstraint`.

O grupo percebeu que testes automatizados cobrem comportamento, mas não aparência nem limpeza, sendo necessário inspeção manual para questões de markup, legibilidade de ícones e resíduos de arquivos como `.claudelaunch.json`. Ao mesmo tempo, a ferramenta de testes foi usada para travar comportamentos sutis (CSRF efetivo nos formulários, redirecionamento correto de anônimos para `login?next=...`, e garantias de que dados não mudam em cenários de 403), reforçando a disciplina de TDD promovida pelos agentes.

Plano de testes e sua execução na prática

Filosofia de testes e TDD com agentes

Todos os planos (`login-cadastro.md`, `catalogo-item.md`, `review-e-conta.md`) seguem um padrão explícito de TDD: escrever testes que falham, rodar `manage.py test` para confirmar a falha, implementar o mínimo necessário, rodar novamente e garantir que os testes passem antes de avançar. Os agentes foram instruídos a usar `manage.py test` com verbosidade e, em alguns casos, `Client(enforce_csrf_checks=True)` para assegurar que a suíte não passasse por acidente ignorando CSRF.

O plano de testes foi desenhado para cobrir todos os critérios explícitos da seção 5 do documento original que permaneceram no escopo, com tabelas indicando quantos testes por fatia, quais comportamentos são travados e que critérios ficam sem implementação e sem teste, exigindo correção futura no .docx.

Cobertura de autenticação e rotas protegidas

Na fatia de autenticação, sete testes planejados cobriam cadastro válido com senha hashada, email duplicado, email vazio, login válido redirecionando para catalogo, login inválido sem sessão, logout via POST, e logout via GET retornando 405, garantindo que sessões fossem criadas e encerradas da forma esperada. A implementação acabou entregando dez testes, adicionando casos naturais como GET em login e cadastro retornando 200 e verificando que login inválido não cria sessão.

Na fatia do catálogo de itens, o plano em `catalogo-item.md` adicionou testes para `LoginRequiredMixin` em todas as rotas, criação de itens com `criadopor` forjado ignorado em favor de `request.user`, upload de capa gravando arquivos em diretório temporário com `overridesettings(MEDIA_ROOT=...)`, e autorização por `DonoOuAdminMixin` com 403 para usuários comuns em editar/excluir.

Cobertura de reviews, CASCADE e exclusão de conta

Na fatia final, o plano em `review-e-conta.md` especificou testes para o modelo `Review` (bordas de nota 0 e 10 válidas, nota 11 inválida, opinião vazia aceita, `UniqueConstraint` barrando duplicatas via `IntegrityError` dentro de `transaction.atomic`) e para as views de criação, edição e exclusão. Os testes de criação verificam que `usuario` e `item` são preenchidos pela view, que campos forjados no POST são ignorados, que segunda tentativa em `avaliar` redireciona para editar em vez de gerar 500, e que o dono do item pode avaliar o próprio item.

Para exclusão de conta, os testes em `usuariotests.py` garantem que anônimos em `contaexcluir` são redirecionados para login, que GET mostra a contagem de itens próprios e de reviews de terceiros que serão destruídas, e que POST apaga a conta, encerra a sessão e executa CASCADE sobre itens e reviews, incluindo o caso em que um usuário perde a review dele quando outro apaga a conta.

Resultados e estado final da suíte

O resumo da sessão registra a evolução da contagem de testes: 10 na fatia de autenticação, 23 após a fatia 1 do catálogo, 33 ao fechar CRUD de item, 55 ao concluir review e exclusão de conta, e 62 após ajustes finais incluindo tela de 404 e outras melhorias. Em todos os momentos, `manage.py check` foi usado para garantir ausência de migrações pendentes e problemas estruturais, e o CI configurado no início passou a rodar uma suíte não trivial, em vez de apenas "Found 0 tests".

Dois critérios do documento original permanecem sem implementação e sem teste: "remoção de item em uso bloqueada" (derrubado conscientemente pelo CASCADE) e "usuário comenta a preferência de outro usuário" (eliminado com a remoção do modelo Comentario), e essas lacunas são reconhecidas como pontos que exigem ajuste na especificação escrita ou justificativa clara no relatório.

Conclusões sobre o processo com agentes, experiência e testes

O desenvolvimento da Gostoteca mostra que agentes podem estruturar specs, planos TDD e código com alta produtividade, mas que a qualidade final depende de revisões humanas, tanto para alinhamento com o documento de requisitos quanto para aspectos visuais, de escopo e de ética de dados (como o impacto de CASCADE em dados de terceiros). A experiência do grupo evidencia a importância de interromper o fluxo automatizado quando há discrepâncias, fazer perguntas em vez de assumir, e registrar todas as divergências de forma transparente para futuras avaliações acadêmicas.

O plano de testes funcionou bem na prática: guiou a implementação por agentes, revelou armadilhas de framework (POST-only em logout, validação parcial em `ModelForm` com campos ausentes, necessidade de `transaction.atomic`), e produziu uma suíte de mais de sessenta testes cobrindo autenticação, catálogo, reviews e exclusão de conta. Ao mesmo tempo, mostrou os limites da testabilidade automatizada para aspectos de markup e experiência visual, reforçando que inspeção manual e senso crítico do grupo permanecem essenciais em projetos desenvolvidos com apoio de agentes.